

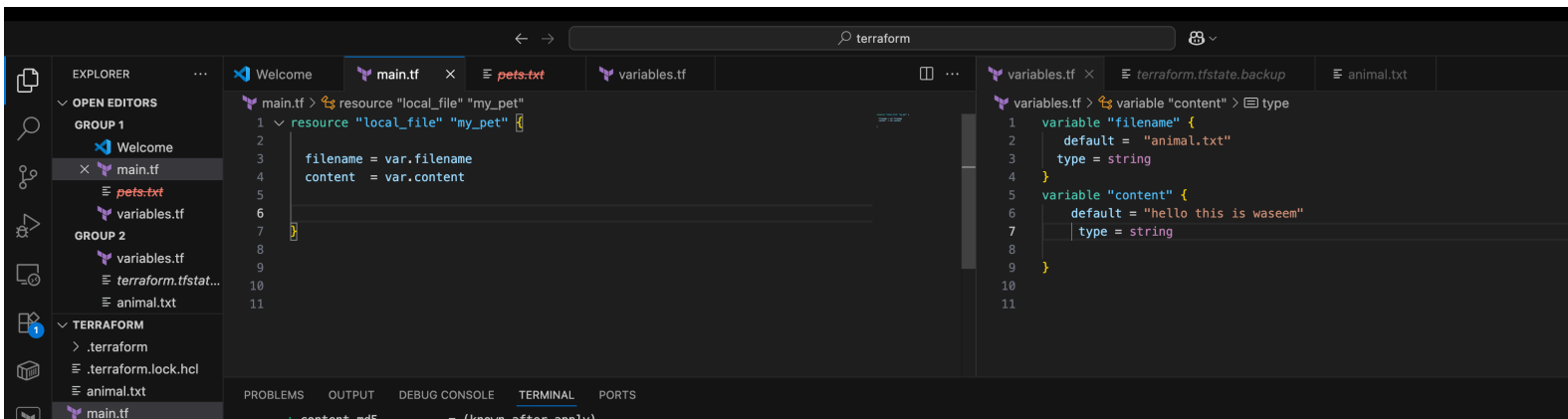
Terraform 3

1. Watch the Terraform-03 video.
2. Execute the script shown in the video.

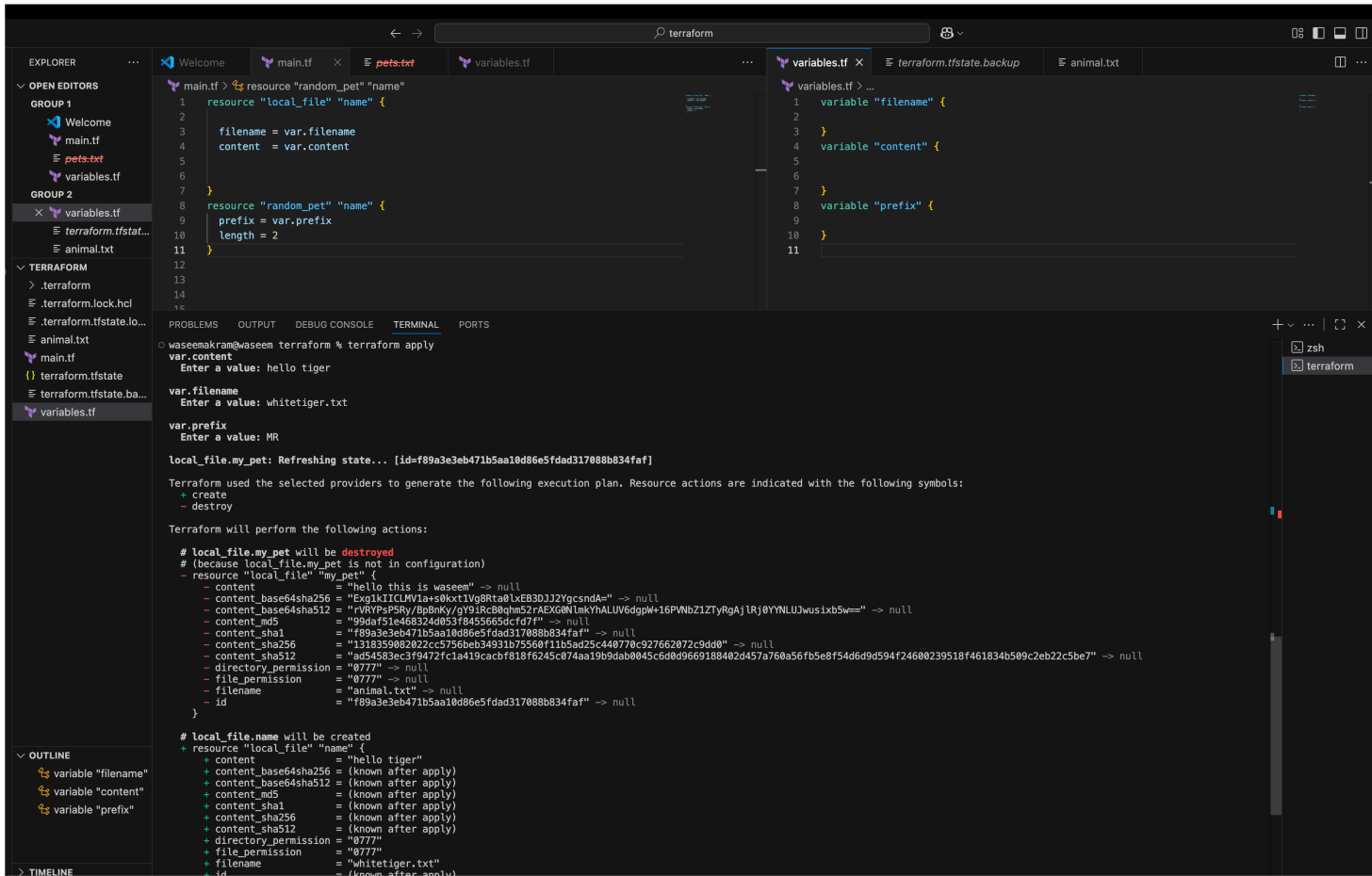
Using of variables:

=====

1) By using variables.tf file



2) By using interactive mode (This will get activated if we dont pass default value in variable.tf file)



```
waseemakram@waseem terraform % terraform apply
var.content
Enter a value: hello tiger

var.filename
Enter a value: whitetiger.txt

var.prefix
Enter a value: MR

local_file.my_pet: Refreshing state... [id=f89a3e3eb471b5aa10d86e5fdad317088b834faf]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

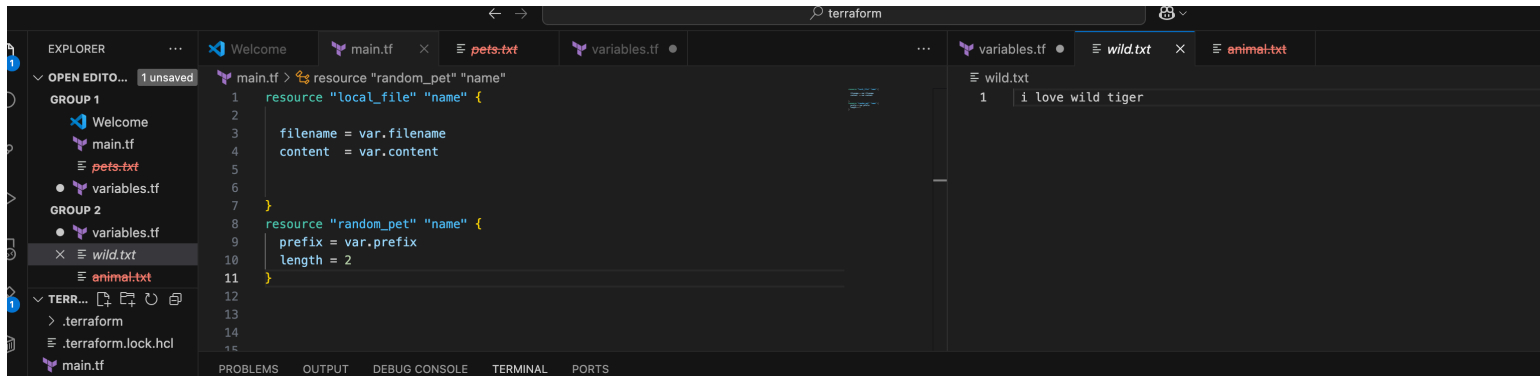
# local_file.my_pet will be destroyed
# (because local_file.my_pet is not in configuration)
- resource "local_file" "my_pet" {
  content = "hello this is waseem" -> null
  content_base64sha256 = "Exg1kIICLMV1a+s0kxt1Vg8Rta0LxE83DJJ2YgcsndA=" -> null
  content_base64sha512 = "rVRYPsP5Ry/BpBnKy/gY9iRcB0qhm52rAEXG0NlmyYhALUV6dgpW+16PVNbZ1ZTyRgAj1Rj0YNNLUJwusixb5w==" -> null
  content_md5 = "99daf51e468324d053f8455665dcfd7f" -> null
  content_sha1 = "f89a3e3eb471b5aa10d86e5fdad317088b834faf" -> null
  content_sha256 = "1318359082022cc5756beb34931b75560f11b5ad25c440770c927662072c9dd0" -> null
  content_sha512 = "ad54583ec3f9472f1ca419cacbf818f6245c074aa19b9dab0045c6d0d9669188402d457a760a56fb5e8f54d6d9d594f24600239518f461834b509c2eb22c5be7" -> null
  directory_permission = "0777" -> null
  file_permission = "0777" -> null
  filename = "animal.txt" -> null
  id = "f89a3e3eb471b5aa10d86e5fdad317088b834faf" -> null
}

# local_file.name will be created
+ resource "local_file" "name" {
  content = "hello tiger"
  content_base64sha256 = (known after apply)
  content_base64sha512 = (known after apply)
  content_md5 = (known after apply)
  content_sha1 = (known after apply)
  content_sha256 = (known after apply)
  content_sha512 = (known after apply)
  directory_permission = "0777"
  file_permission = "0777"
  filename = "whitetiger.txt"
  id = (known after apply)
}
```

3) Command line flags

-->

terraform apply -var "filename=wild.txt" -var "prefix=MR" -var "content= i love wild tigers"



```
waseemakram@waseem terraform % terraform apply -var "filename=wild.txt" -var "prefix=MR" -var "content= i love wild tiger"

local_file.name: Refreshing state... [id=562a1098d3e8a3470649ed3f47a8488bfe8ada45]
random_pet.name: Refreshing state... [id=MR-optimal-snapper]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.name must be replaced
-/+ resource "local_file" "name" {
  ~ content      = "hello tiger" -> " i love wild tiger" # forces replacement
  ~ content_base64sha256 = "pRxBC6D14/FqIXMDRqSTi2xGDDGnKU1Jw/8DShqu7u0=" -> (known after apply)
  ~ content_base64sha512 = "nWgXk+45kkI3sb0dsYUIGUp2bQ06SmmVY3y07/2Iwduy2C+ELBSZ8FqR9vC04FmG0iA3wM1d9uBgM3PjTPhcVw==" -> (known after apply)
  ~ content_md5      = "6bd9329556e5cd27ab801f6c1a4fb6e2" -> (known after apply)
  ~ content_sha1      = "562a1098d3e8a3470649ed3f47a8488bfe8ada45" -> (known after apply)
  ~ content_sha256     = "a51c410ba0f5e3f16a23130346ab138b6c460c31a7294d49c3ff034a1aae4" -> (known after apply)
  ~ content_sha512     = "9d68172bee39924237b1b39db18508194a766d0d3a4a6c15637cb4effd88c1dbb2d82f842c1499f05a91f6f08ee05986d22037c0cd5df6e0603373e34cf85c57" -> (known after apply)
  ~ filename         = "whitetiger.txt" -> "wild.txt" # forces replacement
  ~ id               = "562a1098d3e8a3470649ed3f47a8488bfe8ada45" -> (known after apply)
}
# (2 unchanged attributes hidden)

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.name: Destroying... [id=562a1098d3e8a3470649ed3f47a8488bfe8ada45]
local_file.name: Destruction complete after 0s
local_file.name: Creating...
local_file.name: Creation complete after 0s [id=3fde2e0b5d5528ef88db2502236d450252e22253]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform %
```

4) Environment variables

Mac (zsh) equivalent commands

Set variables like this:

```
export TF_VAR_filename="park.txt"
export TF_VAR_content="Dinosaur"
export TF_VAR_prefix="MR"
```

Then run:

```
terraform apply
```

```
● waseemakram@waseem terraform % export TF_VAR_filename="park.txt"
● waseemakram@waseem terraform % export TF_VAR_content="Dinosaur"
● waseemakram@waseem terraform % export TF_VAR_prefix="MR"
● waseemakram@waseem terraform % terraform apply
random_pet.name: Refreshing state... [id=MR-optimal-snapper]
local_file.name: Refreshing state... [id=3fde2e0b5d5528ef88db2502236d450252e22253]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.name must be replaced
-/+ resource "local_file" "name" {
  ~ content           = " i love wild tiger" -> "Dinosaur" # forces replacement
  ~ content_base64sha256 = "GPskdukqjBA1ah94nLZORXtCN8RKJvtwRDD+YOCTYMY=" -> (known after apply)
  ~ content_base64sha512 = "09IDCVJ0bqpsAB3fsDUbW6WiCGWVKQzUNhX5b5NjGcoZbt11SW/EeU8qLh637XSK/q/8YA62Rg4e82AIdT0wEg==" -> (known after apply)
  ~ content_md5         = "58b59dd15304f712231f6494df5f0118" -> (known after apply)
  ~ content_sha1        = "3fde2e0b5d5528ef88db2502236d450252e22253" -> (known after apply)
  ~ content_sha256      = "18fb2476e92a8c10356a1f789e564e457b4237c44a26fb704430fe60e09360c6" -> (known after apply)
  ~ content_sha512      = "3bd2030952746eaa6c001ddfb0351b5ba5a2086595290cd43615f96f936319ca196edd62496fc4794f2a2e1eb7ed748afeaffc600eb6460e1ef36008753d3012" -> (known after apply)
  ~ filename           = "wild.txt" -> "park.txt" # forces replacement
  ~ id                 = "3fde2e0b5d5528ef88db2502236d450252e22253" -> (known after apply)
  # (2 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

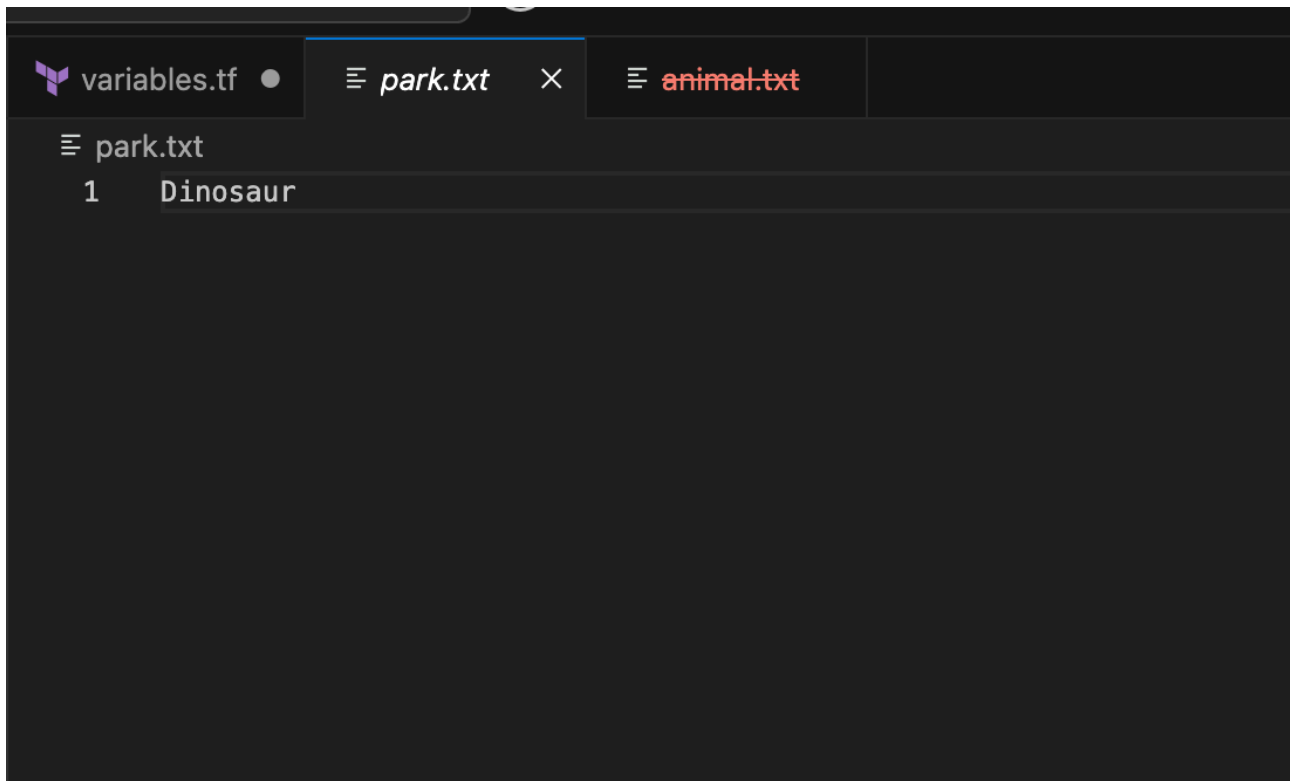
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.name: Destroying... [id=3fde2e0b5d5528ef88db2502236d450252e22253]
local_file.name: Destruction complete after 0s
local_file.name: Creating...
local_file.name: Creation complete after 0s [id=ede5ac98c1c275a0650e2dc3136e975a181a865c]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
○ waseemakram@waseem terraform %
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Plain Text



5) variable definition file (Should be end with terraform.tfvars/
terraform.tfvars.json)

--

> for automatically loaded file name *.auto.tfvars/*.auto.tfvars.json

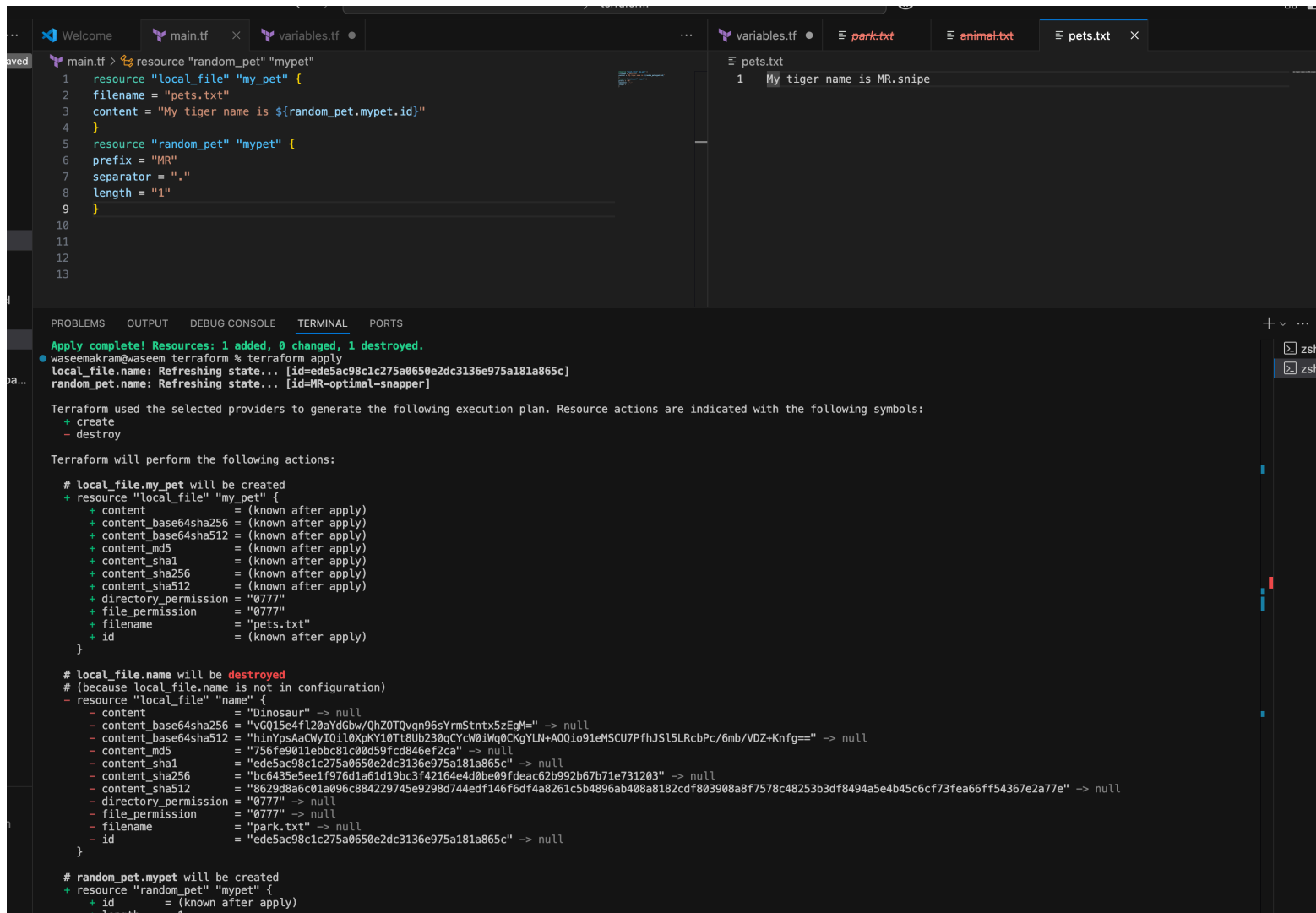
--

> if we are saving the file with other name like variable.tfvars then we need to pass this in

CLI

--

> terraform apply -var-file variable.tfvars



The screenshot shows a VS Code editor with several files open: `main.tf`, `variables.tf`, `park.txt`, `animal.txt`, and `pets.txt`. The `main.tf` file contains Terraform configuration for a `local_file` resource named `my_pet` and a `random_pet` resource named `mypet`. The `local_file` resource is configured with `filename = "pets.txt"` and `content = "My tiger name is ${random_pet.mypet.id}"`. The `random_pet` resource is configured with `prefix = "MR"`, `separator = "."`, and `length = "1"`. The `pets.txt` file shows the output of the `terraform apply` command, displaying the generated content: `1 My tiger name is MR.snipe`. The terminal window at the bottom shows the output of the `terraform apply` command, indicating that the resources were successfully created and the state was refreshed.

```
main.tf > resource "random_pet" "mypet"
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content = "My tiger name is ${random_pet.mypet.id}"
4 }
5 resource "random_pet" "mypet" {
6   prefix = "MR"
7   separator = "."
8   length = "1"
9 }
10
11
12
13

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform % terraform apply
local_file.name: Refreshing state... [id=ede5ac98c1c275a0650e2dc3136e975a181a865c]
random_pet.name: Refreshing state... [id=MR-optimal-snapper]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+   content          = (known after apply)
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5       = (known after apply)
+   content_sha1      = (known after apply)
+   content_sha256    = (known after apply)
+   content_sha512    = (known after apply)
+   directory_permission = "0777"
+   file_permission   = "0777"
+   filename          = "pets.txt"
+   id                = (known after apply)
}

# local_file.name will be destroyed
# (because local_file.name is not in configuration)
- resource "local_file" "name" {
-   content          = "Dinosaur" -> null
-   content_base64sha256 = "vGQ15e4f120aYdGbw/QhZ0TQvgn96sYrmStntx5zEgM=" -> null
-   content_base64sha512 = "hinYpsAaCwyIQil0XpKY10Tt8Ub230qCYcW01Wq0CKgYLN+A0Qio91eMSCU7PfhJS15LRcbPc/6mb/VDZ+Knfg==" -> null
-   content_md5       = "756fe9011ebbc81c00d59fcd846ef2ca" -> null
-   content_sha1      = "ede5ac98c1c275a0650e2dc3136e975a181a865c" -> null
-   content_sha256    = "bc6435e5e1f976d1a61d19bc3f42164e4d0be09fdeac62b992b67b71e731203" -> null
-   content_sha512    = "8629d8a6c01a096c884229745e9298d744edf146f6df4a8261c5b4896ab408a8182cdf803908a8f7578c48253b3df8494a5e4b45c6cf73fea66ff54367e2a77e" -> null
-   directory_permission = "0777" -> null
-   file_permission   = "0777" -> null
-   filename          = "park.txt" -> null
-   id                = "ede5ac98c1c275a0650e2dc3136e975a181a865c" -> null
}

# random_pet.mypet will be created
+ resource "random_pet" "mypet" {
+   id          = (known after apply)
+   length      = 1
+ }
```

Output variable:

These are used to display the output of the resources.

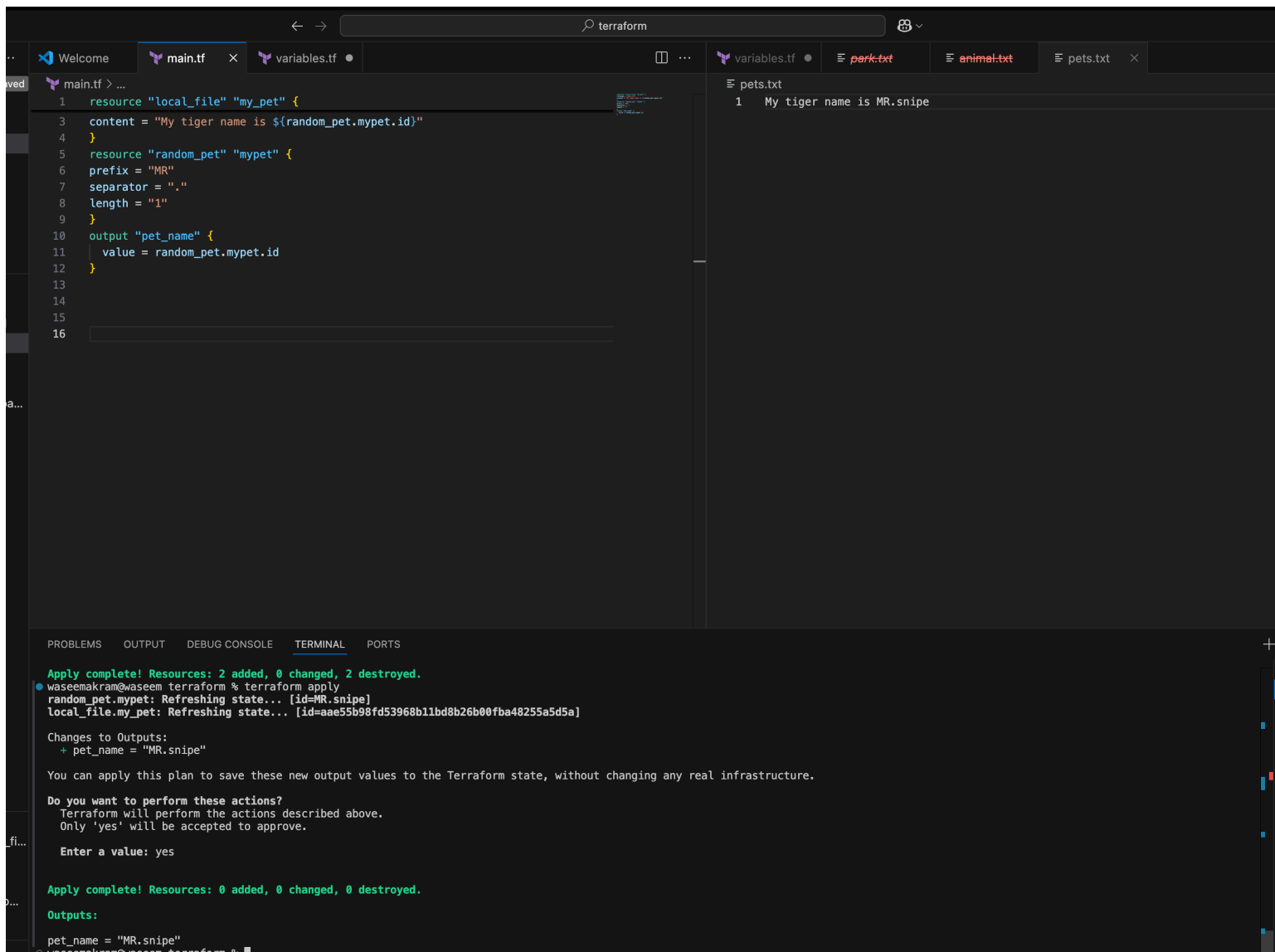
Ex:

```
resource "random_pet" "mypet" {
  prefix = "MR"
  separator = "."
  length = "1"
}
```

```
output my-pet {
  value = random_pet.mypet.id
}
```

```
description = optional name
}
```

when we use terraform apply we can see the id as output.
we can use terraform output command to see the output of the resource.



The screenshot shows a code editor with several files open: `main.tf`, `variables.tf`, `park.txt`, `animal.txt`, and `pets.txt`. The `main.tf` file contains the following Terraform configuration:

```
1 resource "local_file" "my_pet" {
2   content = "My tiger name is ${random_pet.mypet.id}"
3 }
4
5 resource "random_pet" "mypet" {
6   prefix = "MR"
7   separator = "."
8   length = "1"
9 }
10 output "pet_name" {
11   value = random_pet.mypet.id
12 }
13
14
15
16
```

The `pets.txt` file contains the output of the `terraform apply` command:

```
1 My tiger name is MR.snipe
```

The terminal output at the bottom shows the execution of `terraform apply` and the resulting output:

```
Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
waseemakram@waseem terraform % terraform apply
random_pet.mypet: Refreshing state... [id=MR.snipe]
local_file.my_pet: Refreshing state... [id=aae55b98fd53968b11bd8b26b00fba4825a5d5a]

Changes to Outputs:
  pet_name = "MR.snipe"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
pet_name = "MR.snipe"
waseemakram@waseem terraform %
```

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

Outputs:

```
pet_name = "MR.snipe"
```

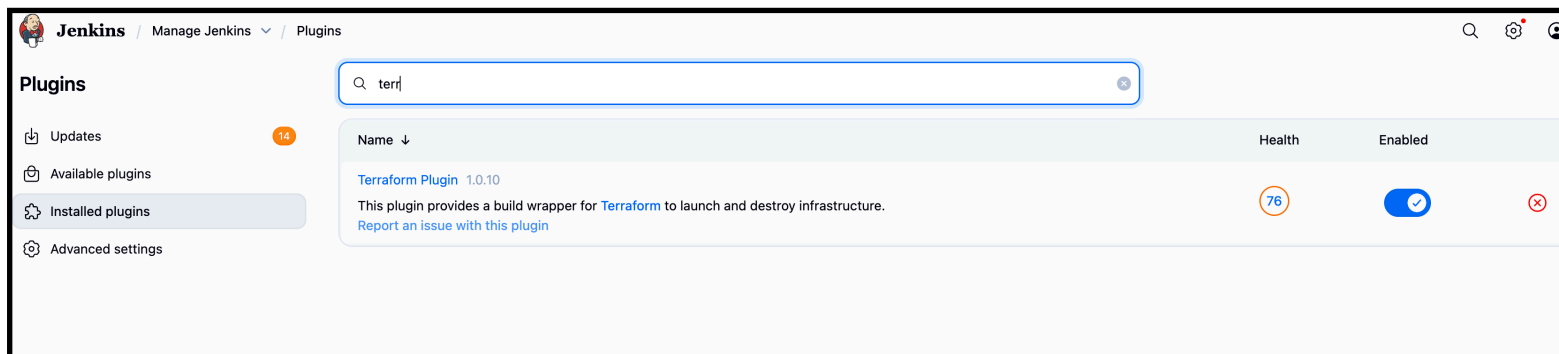
```
waseemakram@waseem terraform %
```

Task

3) Integrate Terraform in Jenkins using the Terraform plugin.

Step 1: Install the Terraform Plugin in Jenkins

1. Go to **Jenkins Dashboard** → **Manage Jenkins** → **Plugins** → **Available Plugins**.
2. Search for **Terraform**.
3. Install **Terraform Plugin**
4. Restart Jenkins if required.



Step 2: Configure Terraform in Jenkins

1. Go to **Manage Jenkins** → **Tools**.
2. Find **Terraform installations**.
3. Click **Add Terraform**:
 - Name: `terraform`
 - Install automatically: ☒ (Jenkins will download it, or you can specify the path if already installed).
4. Save.



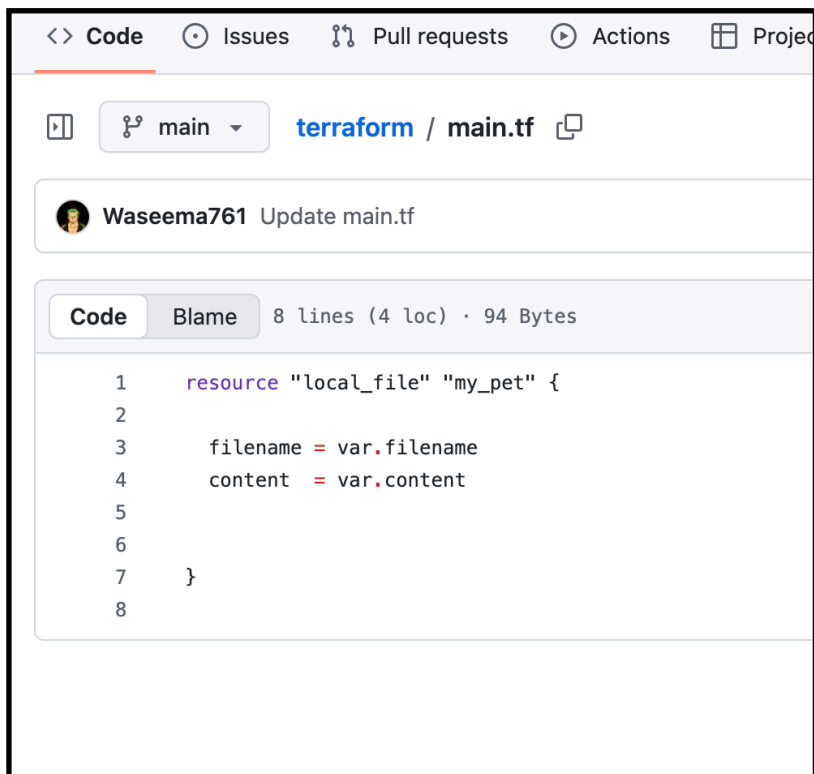
The screenshot shows the 'Terraform installations' configuration page in Jenkins. At the top, there's a tab labeled 'Terraform installations' with a dropdown arrow and an 'Edited' status. Below this is a '+ Add Terraform' button. The main configuration area is titled 'Terraform' and contains a 'Name' field with the value 'terraform'. Below the name field is a checkbox labeled 'Install automatically' which is checked. Underneath this, there's a section titled 'Install from bintray.com' with a 'Version' dropdown menu showing 'Terraform 51203 darwin (amd64)'. At the bottom of this section is a '+ Add Installer' button. The entire configuration area is enclosed in a dashed border with a red 'x' icon in the top right corner. At the very bottom of the page, there are 'Save' and 'Apply' buttons.

Step 3: Create a Jenkins Job (Pipeline or Freestyle)

You can use either **Freestyle** or **Pipeline**.

—> inside GitHub

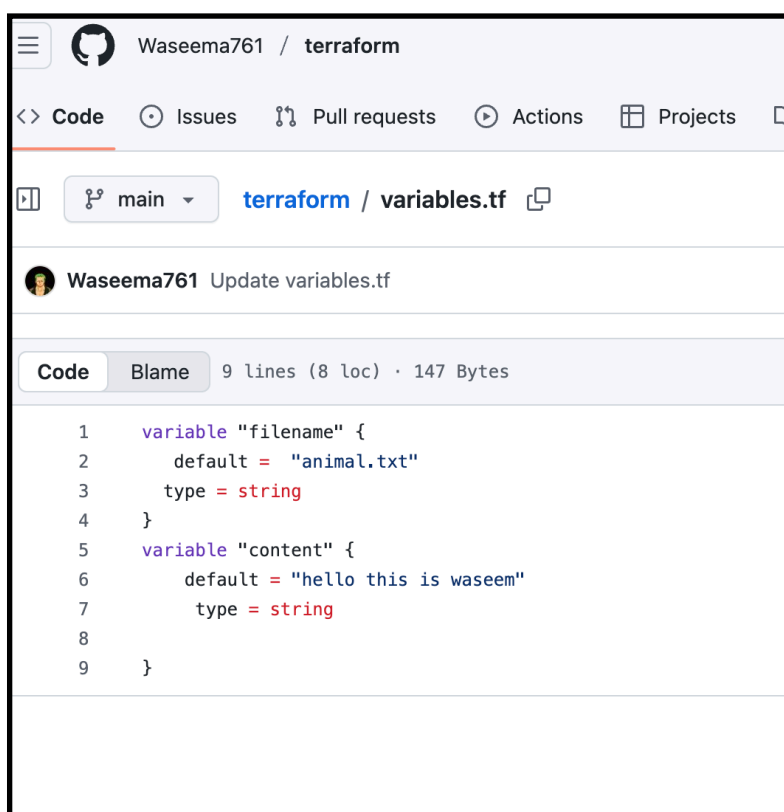
—> add this code in main.tf in your repo file



This screenshot shows a GitHub pull request interface. At the top, there are tabs for 'Code', 'Issues', 'Pull requests', 'Actions', and 'Projects'. Below these, the repository 'terraform' and the file 'main.tf' are displayed. A commit by 'Waseema761' is shown with the message 'Update main.tf'. The 'Code' tab is selected, showing a diff for 'main.tf' with 8 lines (4 loc) and 94 Bytes. The code defines a resource 'local_file' named 'my_pet' with attributes 'filename' and 'content'.

```
1 resource "local_file" "my_pet" {  
2  
3     filename = var.filename  
4     content  = var.content  
5  
6  
7 }  
8
```

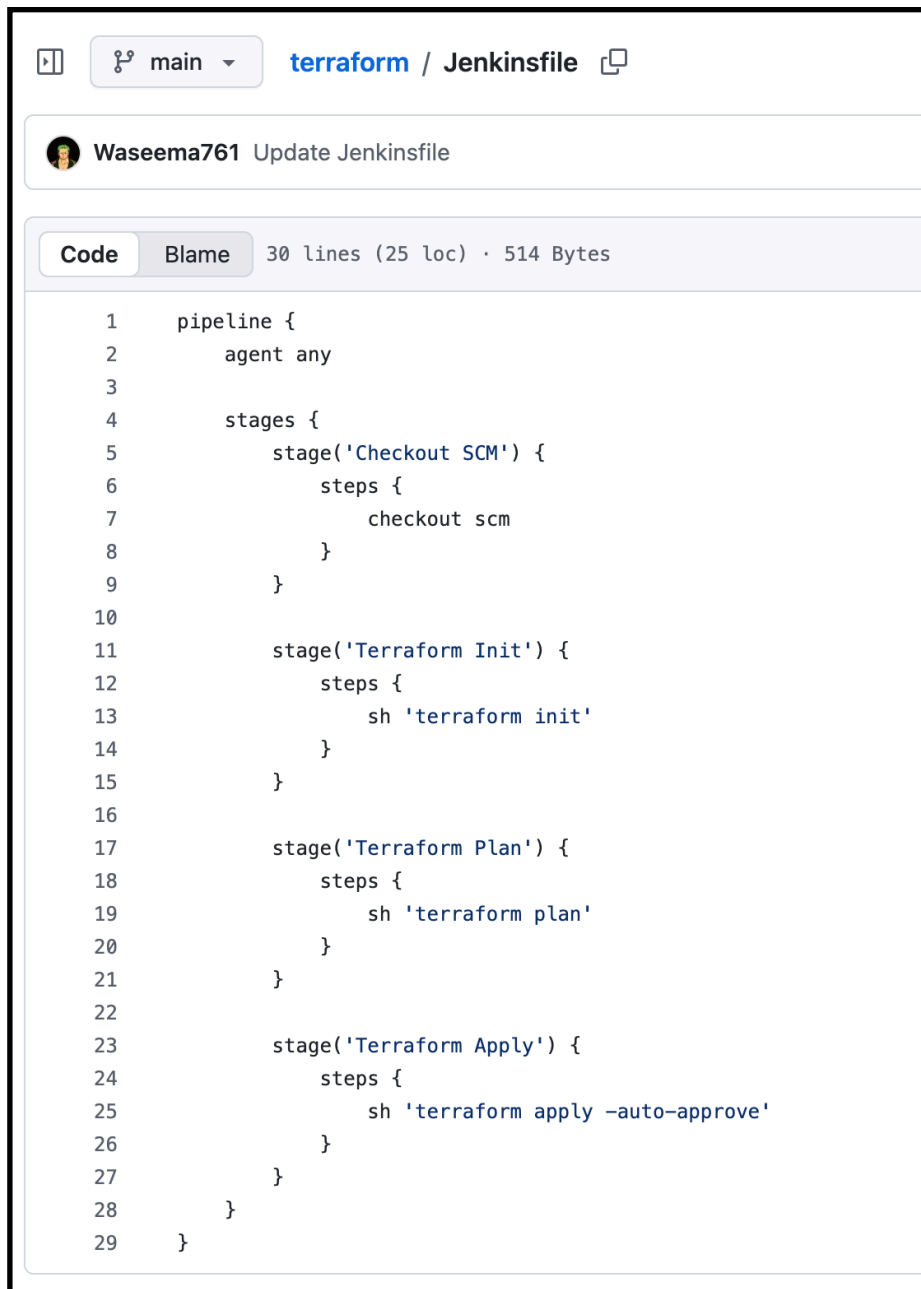
—> add this code in your variables.tf file



This screenshot shows a GitHub pull request interface for the 'variables.tf' file. The repository 'terraform' is shown. A commit by 'Waseema761' is shown with the message 'Update variables.tf'. The 'Code' tab is selected, showing a diff for 'variables.tf' with 9 lines (8 loc) and 147 Bytes. The code defines two variables: 'filename' and 'content'.

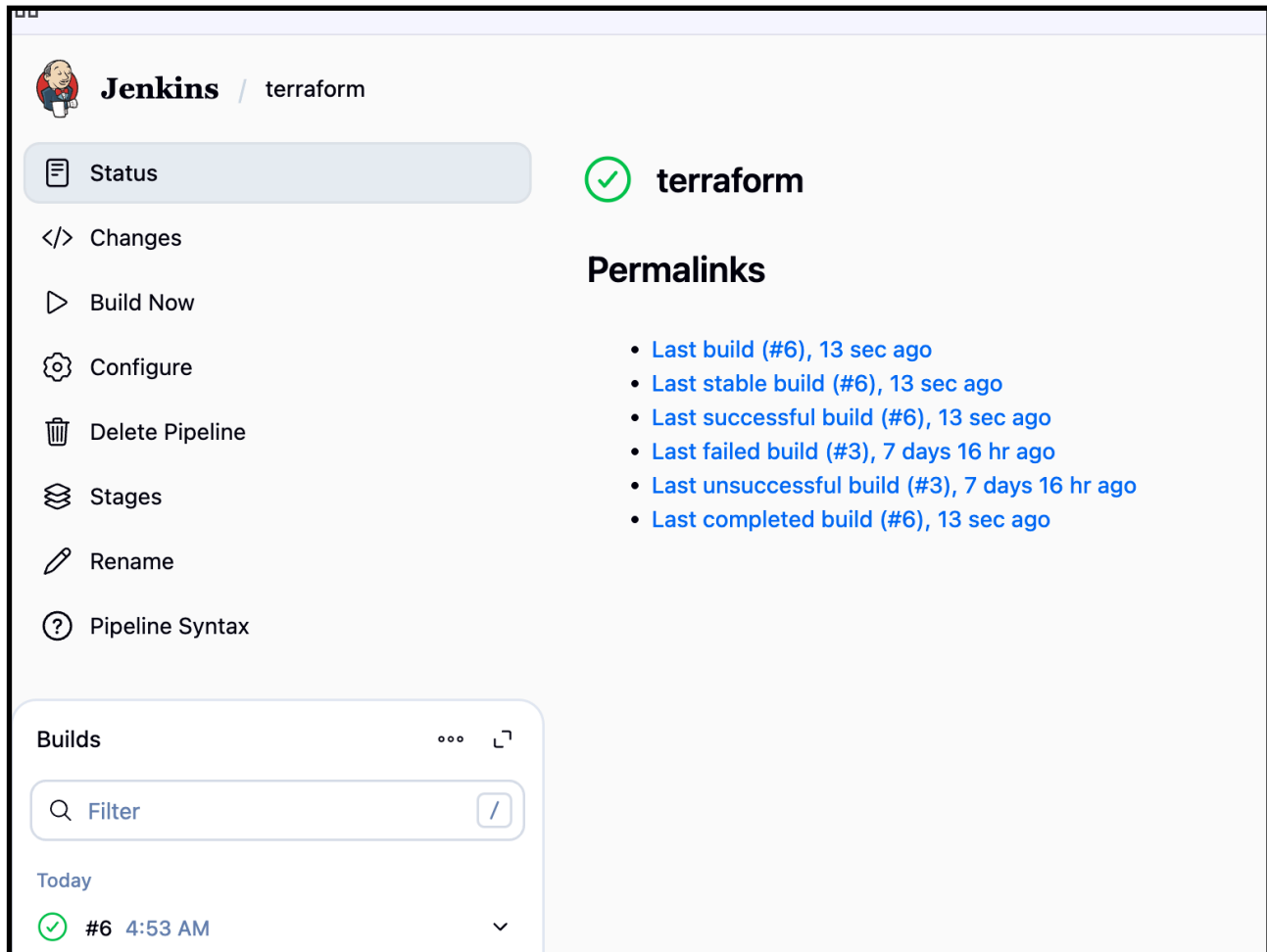
```
1 variable "filename" {  
2     default = "animal.txt"  
3     type = string  
4 }  
5 variable "content" {  
6     default = "hello this is waseem"  
7     type = string  
8 }  
9
```

—> jenkinsfile



```
1  pipeline {
2    agent any
3
4    stages {
5      stage('Checkout SCM') {
6        steps {
7          checkout scm
8        }
9      }
10
11     stage('Terraform Init') {
12       steps {
13         sh 'terraform init'
14       }
15     }
16
17     stage('Terraform Plan') {
18       steps {
19         sh 'terraform plan'
20       }
21     }
22
23     stage('Terraform Apply') {
24       steps {
25         sh 'terraform apply -auto-approve'
26       }
27     }
28   }
29 }
```

Now create a job and build



The screenshot shows the Jenkins web interface for a job named 'terraform'. The interface includes a left sidebar with navigation options: Status (selected), Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, and Pipeline Syntax. The main content area shows the job status as 'terraform' with a green checkmark icon. Below this, there is a 'Permalinks' section with a list of links for various build types and their timestamps. At the bottom, there is a 'Builds' section with a search filter and a list of builds, showing the most recent build as '#6' at '4:53 AM' with a green checkmark icon.

Jenkins / terraform

Status

</> Changes

▶ Build Now

⚙️ Configure

🗑️ Delete Pipeline

📁 Stages

✎ Rename

❓ Pipeline Syntax

terraform

Permalinks

- [Last build \(#6\), 13 sec ago](#)
- [Last stable build \(#6\), 13 sec ago](#)
- [Last successful build \(#6\), 13 sec ago](#)
- [Last failed build \(#3\), 7 days 16 hr ago](#)
- [Last unsuccessful build \(#3\), 7 days 16 hr ago](#)
- [Last completed build \(#6\), 13 sec ago](#)

Builds

Filter

Today

✓ #6 4:53 AM

Check the Jenkins main workspace and cat the file inside the job you can view the content in the file

```
[ec2-user@ip-172-31-65-24 ~]$ cd /var/lib/jenkins/workspace/terraform
[ec2-user@ip-172-31-65-24 terraform]$ ls
Jenkinsfile README.md animal.txt main.tf pets.txt terraform.tfstate terraform.tfstate.backup variables.tf
[ec2-user@ip-172-31-65-24 terraform]$
```

Task 4: Create one Jenkins job using Maven Project for the code below with two stages:

Stage 1: Git clone

Stage 2: Maven Compilation Code: <https://github.com/betawins/java-Working-app.git>

Step-1

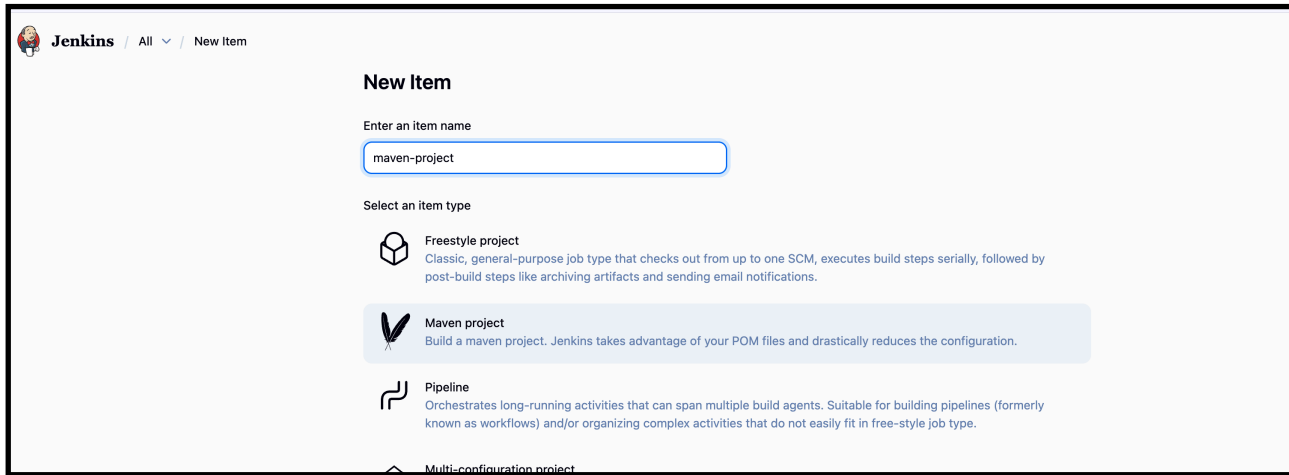
Install the **Maven Integration plugin** from **Manage Jenkins → Plugins**.



Step 2: create a job

Name it : maven-project

Select type : maven project



The image shows the Jenkins 'New Item' page. At the top left is the Jenkins logo and the text 'Jenkins / All / New Item'. The main heading is 'New Item'. Below it is a text input field labeled 'Enter an item name' with the value 'maven-project'. Underneath is a section 'Select an item type' with three options: 'Freestyle project' (described as a classic, general-purpose job type), 'Maven project' (described as building a maven project, highlighted with a blue background), and 'Pipeline' (described as orchestrating long-running activities). A partially visible 'Multi-configuration project' option is at the bottom.

Step-3

Go to **Manage Jenkins → Tool Configuration → Maven**.

Add a Maven installation → check **"Install automatically"** → Jenkins downloads Maven when needed.



The image shows the Jenkins 'Maven' tool configuration page. The title is 'Maven'. There is a 'Name' field with the value 'maven-02'. Below it is a checkbox labeled 'Install automatically' which is checked. Underneath is a section titled 'Install from Apache' with a 'Version' dropdown menu set to '3.9.12'. At the bottom of this section is a '+ Add Installer' button. At the very bottom of the page is a '+ Add Maven' button.

In **Source Code Management**, select **Git** and add your repo URL (<https://github.com/betawins/java-Working-app.git>).

maven-project / Configuration

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/betawins/java-Working-app.git

Credentials ?

- none -

+ Add

Advanced ▾

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

In **Build**, set:

Jenkins / maven-project / Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

+ Add pre-build step

Build

Maven Version

maven-02

Root POM ?

pom.xml

Goals and options ?

clean package

Advanced ▾

- **Goals and options:** clean package
- Jenkins will run Maven using the installed Maven tool.

—> save

—-> now build and check output

 **Jenkins** / maven-project

Status

</> Changes

Workspace

▶ Build Now

⚙️ Configure

🗑️ Delete Maven project

📄 Modules

✎ Rename

Builds

⋮

🔍 Filter

/

Today

🟢 #1 5:11 AM

▼

🟢 maven-project

Permalinks

- [Last build \(#1\), 16 sec ago](#)
- [Last stable build \(#1\), 16 sec ago](#)
- [Last successful build \(#1\), 16 sec ago](#)
- [Last completed build \(#1\), 16 sec ago](#)

Maven by default puts the output in the project's **target/** directory.

Jenkins checks out your Git repo into a **workspace**:

/var/lib/jenkins/workspace/Maven/target/


```
[ec2-user@ip-172-31-65-24 terraform]$ cd ..
[ec2-user@ip-172-31-65-24 workspace]$ ls
'MULTI-STAGE PIPELINE'      hiring-app-declarative      job2                        parametrized-job@tmp      private@tmp                scripted-pipeline
'MULTI-STAGE PIPELINE@tmp'  hiring-app-declarative@tmp  job3                      pipeline                  'scripted pipeline'       simplecustomerapp
first-job                  hiring-app-parallel         maven-project             pipeline@tmp              'scripted pipeline@tmp'   simplecustomerapp
hiring-app                 hiring-app-parallel@tmp     parametrized-job          private                   scripted-pipeline1        terraform
[ec2-user@ip-172-31-65-24 workspace]$ cd maven-project/
[ec2-user@ip-172-31-65-24 maven-project]$ ls
Dockerfile  Jenkinsfile  README.md  'Untitled Diagram.drawio'  jenkinsfile-cicd  pom.xml  src  target
[ec2-user@ip-172-31-65-24 maven-project]$ cd target/
[ec2-user@ip-172-31-65-24 target]$ ls
hiring  hiring.war  maven-archiver
[ec2-user@ip-172-31-65-24 target]$ cd hiring/
[ec2-user@ip-172-31-65-24 hiring]$ ls
META-INF  WEB-INF  index.jsp
[ec2-user@ip-172-31-65-24 hiring]$
```

Task 5:

1. Use the same code and create a **parameterized job** in Jenkins with:
2. **Stage 1:** Git clone

Stage 2: Maven Compilation **Code:** <https://github.com/betawins/java-Working-app.git>

Step 1: Create a Parameterized Job

1. Go to **Jenkins Dashboard** → **New Item**
2. Choose **Pipeline** → give a name → click OK

New Item

Enter an item name

parameterized-job2

Select an item type

- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

In the job config:

- Check **“This project is parameterized”**
- Add parameters:
 - **String Parameter** → Name: `BRANCH_NAME`, Default: `main`, Description: `Git branch to build`
 - **String Parameter** → Name: `MAVEN_GOAL`, Default: `clean package`, Description: `Maven goal to run`

☒ This project is parameterized ?

String Parameter ?

Name ?

BRANCH_NAME

Default Value ?

main

Description ?

git branch install

Plain text [Preview](#)

☐ Trim the string ?

String Parameter ?

Name ?

MAVEN_GOAL

Default Value ?

clean package

Description ?

add clean packages

Plain text [Preview](#)

☐ Trim the string ?

Step 2: Add Scripted Pipeline Code

```
node {  
    def mvnHome = tool name: 'MVN_HOME', type: 'maven'  
    env.PATH = "${mvnHome}/bin:${env.PATH}"  
  
    stage('Git Clone') {  
        git branch: "${BRANCH_NAME}", url: 'https://github.com/  
betawins/java-Working-app.git'  
    }  
  
    stage('Maven Compilation') {  
        sh "mvn ${MAVEN_GOAL}"  
    }  
  
    stage('Archive Package') {  
        // Fix: WAR instead of JAR  
        archiveArtifacts artifacts: 'target/*.war', fingerprint: true  
    }  
}
```

Definition

Pipeline script

Script ?

```
1 node {
2   def mvnHome = tool name: 'MVN_HOME', type: 'maven'
3   env.PATH = "${mvnHome}/bin:${env.PATH}"
4
5   stage('Git Clone') {
6     git branch: "${BRANCH_NAME}", url: 'https://github.com/betawins/java-Working-app.git'
7   }
8
9   stage('Maven Compilation') {
10    sh "mvn ${MAVEN_GOAL}"
11  }
12
13  stage('Archive Package') {
14    // Fix: WAR instead of JAR
15    archiveArtifacts artifacts: 'target/*.war', fingerprint: true
16  }
17 }
```

☒ Use Groovy Sandbox ?

Save and build with parameters

 **Jenkins**

/ parametrized-job2

Status

<> Changes

Build with Parameters

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

Builds

Filter

Pipeline parametrized-job2

This build requires parameters:

BRANCH_NAME

git branch install

main

MAVEN_GOAL

add clean packages

clean package

Build

Cancel

 **Jenkins** / parametrized-job2

Status

Changes

Build with Parameters

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

 **parametrized-job2**

 **Last Successful Artifacts**

 hiring.war 1.66 KiB  view

Permalinks

- [Last build \(#3\), 10 min ago](#)
- [Last stable build \(#3\), 10 min ago](#)
- [Last successful build \(#3\), 10 min ago](#)
- [Last failed build \(#2\), 11 min ago](#)
- [Last unsuccessful build \(#2\), 11 min ago](#)
- [Last completed build \(#3\), 10 min ago](#)

Builds

Filter

Today

 #3 5:55 AM

 #2 5:54 AM

—> check the pipeline stages

 < #3

 Rerun

Manually run by waseem akram

Started 11 min ago

Queued 2 ms

Took 4 sec

Artifacts

Graph

Start

 Git Clone

 Maven Compilation

 Archive Package

End

Search

 Archive Package 89ms

 Started 11m ago

 Jenkins

 Archive the artifacts

0 Archiving artifacts

1 Recording fingerprints

```
[ec2-user@ip-172-31-65-24 ~]$ cd /var/lib/jenkins/workspace/parametirized-job2
[ec2-user@ip-172-31-65-24 parametrized-job2]$ ls
Dockerfile  Jenkinsfile  README.md  'Untitled Diagram.drawio'  jenkinsfile-cicd  pom.xml  src  target
[ec2-user@ip-172-31-65-24 parametrized-job2]$ cd target/
[ec2-user@ip-172-31-65-24 target]$ ls
hiring  hiring.war  maven-archiver
[ec2-user@ip-172-31-65-24 target]$
```

Task :

6.)What are the global variables in Jenkins?

In Jenkins Pipelines, **Global Variables** are special built-in environment variables and objects that Jenkins automatically provides. You don't need to define them — they are available everywhere in your pipeline.

Job & Build Info

- BUILD_ID → Unique build ID (timestamp)
- BUILD_NUMBER → Build number (incremental)
- BUILD_TAG → jenkins-\${JOB_NAME}-\${BUILD_NUMBER}
- JOB_NAME → Name of the Jenkins job
- JOB_BASE_NAME → Job name without folder path
- BUILD_URL → URL of the build in Jenkins
- JOB_URL → URL of the job

Node/Workspace Variables

- NODE_NAME → Name of the agent node running the build
- WORKSPACE → Path to the job's workspace directory

Git/SCM Variables (when using git step)

- GIT_COMMIT → Commit hash being built
- GIT_BRANCH → Branch being built
- GIT_URL → Git repo URL

Special Global Objects (Pipeline DSL)

These are not strings, but powerful objects you can call methods on:

- steps → Provides pipeline steps (e.g. sh, git, echo)
- pipeline → DSL for declarative pipelines
- docker → Access Docker agents/containers if plugin is enabled
- input → Request user input

- `credentials()` → Inject stored credentials

Summary

- **env** → Environment variables
- **params** → Build parameters
- **currentBuild** → Build metadata
- **Built-in vars** → `BUILD_NUMBER`, `JOB_NAME`, `WORKSPACE`, `BUILD_URL`, etc.
- **SCM vars** → `GIT_COMMIT`, `GIT_BRANCH`, `GIT_URL`

